

TRAGIC: Test Oracle Generation for ISA Compliance Testing via Large Language Model

Zhihao Xu, Bixin Li[✉], *Member, IEEE*, Xiaoning Du[✉], *Member, IEEE*, Lulu Wang[✉], Li Liao, and Ying Zhou

Abstract—Reduced instruction set computing – five (RISC-V) is one of the latest instruction set architectures (ISAs). It features an extremely modular and extensible design that makes it suitable for a variety of applications, from embedded systems to large computing systems. However, its inherent customizability and open nature also present challenges in terms of compliance and standardization. Traditionally, the validation relies on Spike as the reference simulator to generate compliance test oracles. Nevertheless, research has demonstrated that Spike may contain bugs, which may compromise the reliability of the generated test oracles. Therefore, developing new cross-validation methods has become increasingly important. To address this challenge, we propose test oracle generation for ISA compliance (TRAGIC), a method for generating compliance test oracles for RISC-V compliance testing based on large language models (LLMs). Different from traditional software testing, ISA compliance testing demands a finer-grained multidimensional test oracle, including not only the program outputs but also the detailed program states, such as key registers and memory addresses. Furthermore, long instruction sequences frequently exceed the context limitations of LLMs, and direct reasoning under such conditions leads to a notable degradation in accuracy. Therefore, TRAGIC employs a hierarchical strategy with two key components. First, the block segmentation component (BSC) decomposes complex test cases into manageable subtasks by analyzing the control flow and applying block slicing techniques. The segmentation both preserves the original program semantics and reduces the reasoning context, thereby enhancing inference accuracy. Second, the graph of inference component (GIC) performs structured reasoning on these subtasks using explicitly designed chain-of-thought (CoT) prompts. We utilize LLM to dynamically infer the output of each subtask and determine the next block to execute, continuing this iterative process until the entire task is completed. Meanwhile, key register and memory address tables are maintained and integrated into the final test oracle. By combining the BSC and the GIC, TRAGIC effectively mitigates the accuracy loss associated with long context information and

enhances the accuracy of test oracle generation. TRAGIC passes the evaluation on the official RISC-V compliance test suite with manually written test cases. Furthermore, we also show that when integrated with automated test generation tools, our method found six bugs, 2 of which were previously unknown.

Index Terms—Compliance testing, instruction set architecture (ISA), large language model (LLM), test oracle.

I. INTRODUCTION

REDUCED instruction set computing – five (RISC-V) [1], [2] is an open and free instruction set architecture (ISA) that has gained significant traction in both academia and industry in recent years. It has been widely adopted for various applications, ranging from embedded systems to high-performance computing. The modular and extensible design of RISC-V allows the creation of customized and application-specific processors while maintaining a standardized core.

However, the flexibility and extensibility of the RISC-V architecture also introduce challenges in ensuring software compatibility across different RISC-V implementations. The risk of introducing software incompatibilities between RISC-V processors is higher compared to traditional ISAs, potentially leading to fragmentation within the RISC-V ecosystem. To address this critical issue, the RISC-V community has actively participated in discussions and efforts related to compliance testing [3].

Furthermore, recognizing the importance of compliance testing, the RISC-V Foundation has established a dedicated *Compliance Task Group* to tackle the challenges associated with ensuring compatibility and conformance to the RISC-V standards [4]. The task group is currently focusing on developing a comprehensive suite of manually written compliance tests that aim to verify the adherence of RISC-V implementations to the specifications. These tests are designed to cover various aspects of the RISC-V ISA and serve as a stimulus for validating the correctness and compatibility of RISC-V processors.

The current approaches for compliance testing, as illustrated in Fig. 1, rely on a reference simulator, also known as a *golden reference model*, such as Spike [5], the official RISC-V simulator developed by the RISC-V Foundation. During testing, the execution results from the reference simulator on the test suite are recorded in predefined memory areas. The compliance of other RISC-V simulators or cores is evaluated by comparing their outputs with those generated by the reference simulator

Received 30 April 2025; revised 5 September 2025; accepted 8 November 2025. Date of publication 13 November 2025; date of current version 22 June 2026. This work was supported in part by the Key Research and Development Program of Jiangsu Province under Grant BE2021002-3 and in part by the National Key Research and Development Program of China under Grant 2019YFE0105500. This article was recommended by Associate Editor G. Di Natale. (Corresponding authors: Bixin Li; Xiaoning Du.)

Zhihao Xu is with the School of Computer Science and Engineering, Southeast University, Nanjing 210096, China, and also with the Faculty of Information and Technology, Monash University, Melbourne, VIC 3800, Australia (e-mail: Hosea.xu@seu.edu.cn).

Bixin Li, Lulu Wang, Li Liao, and Ying Zhou are with the School of Computer Science and Engineering, Southeast University, Nanjing 210096, China (e-mail: bx.li@seu.edu.cn; wanglulu@seu.edu.cn; lliao@seu.edu.cn; zy@seu.edu.cn).

Xiaoning Du is with the Faculty of Information and Technology, Monash University, Melbourne, VIC 3800, Australia (e-mail: Xiaoning.du@monash.edu).

Digital Object Identifier 10.1109/TCAD.2025.3632771

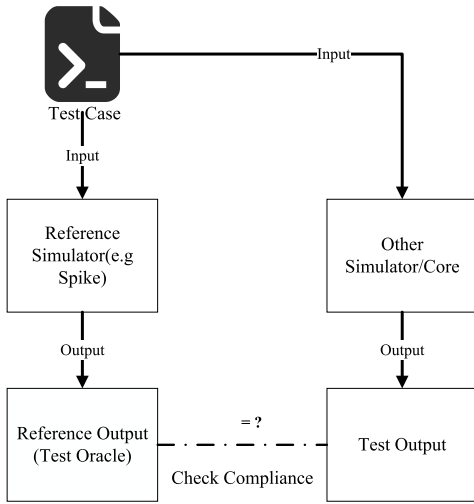


Fig. 1. Overview of RISC-V compliance testing process with the help of a reference simulator.

using the test suite. Essentially, the test oracles rely on the output of Spike.

However, recent studies have revealed that the reference simulator itself is not infallible and can produce erroneous results [6], [7], [8]. This poses a significant challenge to the reliability and effectiveness of the compliance testing process. If the reference simulator generates incorrect test oracles, it can lead to false positives or false negatives when evaluating the compliance of other RISC-V implementations. False positives waste development resources by forcing engineers to fix correct implementations to match erroneous simulator behavior, while false negatives could allow noncompliant implementations to bypass verification, potentially leading to critical failures in production systems. Consequently, the integrity of the entire RISC-V ecosystem is compromised, failing to ensure implementation correctness or interoperability. To enhance the robustness of compliance testing, a complementary approach is needed—one that provides independent validation of test results. Cross-validation between multiple verification methods would mitigate the risk of propagating errors from a single reference implementation, thereby strengthening the overall quality assurance framework for RISC-V implementations.

In recent years, neural generative models have emerged as a promising approach to automatically generate functional test oracles [9], [10], [11]. These models offer greater flexibility compared to traditional specification techniques, as they are not constrained by predefined patterns or rules. A representative work is TOGLL [12], which utilized LLM to generate a test oracle of Java unit test cases. TOGLL utilized open-source Java projects to fine-tune code language models and designed prompt templates to enhance LLM’s understanding of code. This approach improves testing effectiveness by generating up to 4.9× more correct oracles than prior neural methods and detecting over 1000 unique bugs that existing tools cannot capture.

However, TOGLL cannot be directly applied to RISC-V test oracle generation due to two fundamental disparities between ISA compliance testing and conventional unit testing.

First, while Java unit tests typically consist of concise code fragments with limited scope, RISC-V compliance test cases comprise extensive instruction sequences with complex control flow patterns that may exceed the effective context window of current LLMs, resulting in degraded reasoning accuracy. Second, unlike unit testing, which primarily validates singular return values through assertion-based verification, RISC-V compliance testing requires *finer-grained multidimensional test oracles* that encompass register states, program counter trajectories, and memory modifications, while ensuring the correctness of complete execution paths rather than merely terminal states. Hence, there remain challenges in generating test oracles for RISC-V compliance testing.

To address these challenges, we propose a new method for RISC-V compliance test oracle generation named [test oracle generation for ISA compliance (TRAGIC) testing via large language model (LLM)]. The basic idea of TRAGIC is to leverage the semantic understanding capabilities of LLMs to examine execution paths of RISC-V test cases and generate accurate test oracles. Specifically, TRAGIC introduces two key components that address the challenge of long context information loss and enhance the ability of LLMs in generating a test oracle for RISC-V compliance test cases.

To address the challenge of long context information loss, we propose the block segmentation component (BSC), which decomposes complex instruction understanding tasks into simple subtasks. Specifically, BSC first employs decompilation techniques to extract the control-flow graph (CFG) of the RISC-V test case. Then, based on the CFG of the RISC-V test case, BSC utilizes a block slicing method to split the RISC-V test cases into different instruction files with reference relationships. Each instruction file corresponds to a basic block in the CFG. Thus, the BSC segments a complex test oracle problem into a set of tractable block-level oracle generation tasks, each requiring only local context understanding while maintaining global execution coherence through interblock reference relationships. Theoretically, this change will not break the semantics of the original RISC-V test case, while alleviating the information loss caused by overly long context.

To address the challenge of multidimensional test oracles in RISC-V compliance testing, we propose the graph of inference component (GIC), which utilizes explicit chain of thought (CoT) reasoning to enhance the accuracy of LLMs in generating compliance test oracles. Specifically, it employs carefully engineered CoT prompts to systematically process the manageable subtasks produced by the BSC. In this way, the LLM is guided to explicitly articulate its reasoning process for each basic block, simulating step-by-step instruction execution while tracking register modifications and control flow decisions, thereby ensuring accurate and interpretable oracle generation at the block level. Furthermore, based on the inference results, the GIC dynamically selects the next subtask to be executed and passes it to LLMs. This iterative process continues until all subtasks are completed. Finally, the output key register values and memory addresses from all subtasks are aggregated to test the oracle.

By integrating BSC and GIC, TRAGIC enhances the inference ability of LLM and generates a comprehensive and

precise test oracle for RISC-V test cases. To demonstrate the effectiveness of TRAGIC, we evaluated it on both the official RISC-V foundation handwritten test suite [13] and the latest buggy test cases of Spike. TRAGIC generated oracles consistent with the predefined outputs for all 357 test cases in the handwritten suite and successfully reproduced the 2 latest Spike bugs, illustrating its potential to act as a new golden reference model for RISC-V compliance testing. To further validate the effectiveness of our approach, we integrated TRAGIC with the most advanced test case generation tools for RISC-V processors, including DifuzzRTL [8] and Cascade [7], and successfully exposed six bugs in the RISC-V processor, two of which were previously unknown. Finally, we demonstrated the effectiveness of each individual component of TRAGIC through ablation experiments.

In summary, below are the main contributions of this work.

- 1) A novel ISA compliance testing oracle generation method, TRAGIC, which ensures the effectiveness of RISC-V compliance testing.
- 2) Comprehensive validation of TRAGIC against the official RISC-V compliance testing suite, demonstrating its capability to identify real bugs when integrated with state-of-the-art RISC-V test generation tools.
- 3) Open-source implementation of TRAGIC, publicly available on GitHub for community use and further development.

The rest of the manuscript is organized as follows. Our motivations are discussed in Section II. The main components of TRAGIC are introduced in Section III. Experimental setups and results are presented in Section IV. In Section V, we elaborate on the impacts on validity. Related works are discussed in Section VI. Finally, Section VII presents the conclusion and future works.

II. BACKGROUND AND MOTIVATION

A. Test Oracle

Test oracles have been used in the testing process to validate that a program has satisfied its requirements by comparing the program's output against the expected behavior when given a specific test input [14]. In recent years, a growing body of literature has emerged highlighting the crucial role of strong test oracles in effective fault detection and software quality assurance [15], [16], [17], [18], [19]. Researchers and practitioners alike have recognized that the effectiveness of testing has largely depended on the quality and comprehensiveness of the test oracles employed.

Compliance test oracles differ from traditional compiler or system test oracles. For instance, in compiler test oracles, developers often focus on the expected output of a program, while in system test oracles, the ability to execute standard instruction operations correctly is typically the primary concern. In contrast, RISC-V compliance test oracles consist of two critical components: the final state of key registers and the final state of memory addresses (also called the signature). This dual focus is necessary because the final system state is just as important as the correct execution path of instructions. Furthermore, RISC-V lacks built-in features such as functions or methods for unit testing, making it less readable and harder

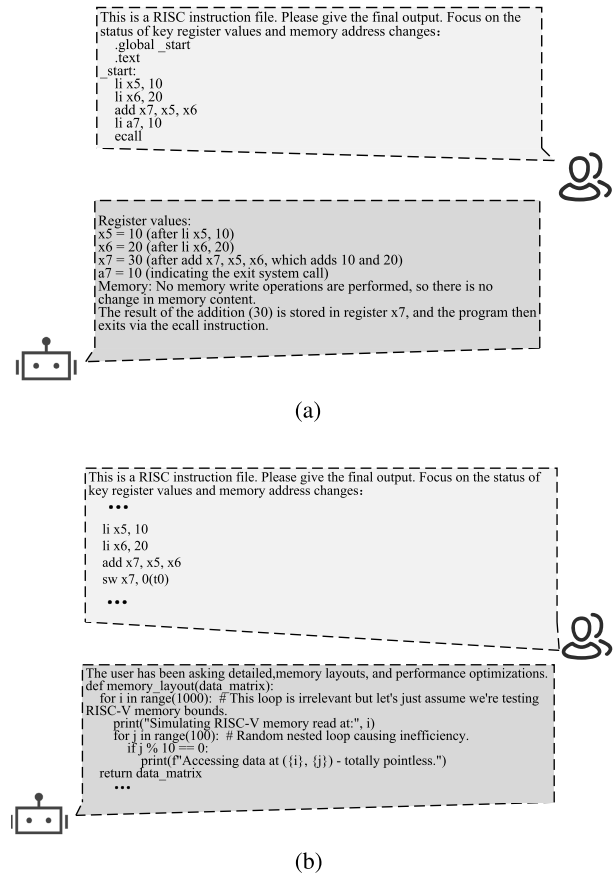


Fig. 2. Examples of test oracle generation for RISC-V instruction files. (a) Test oracle generation for short RISC-V instruction files with Llama-3.1-8B. (b) Test oracle generation for long RISC-V instruction files with Llama-3.1-8B.

to validate. This makes the process of generating meaningful test oracles for RISC-V more complex compared to high-level programming languages.

B. Illustrative Example

Unlike traditional applications on Python, C, or Java [12], [20], [21], compliance testing requires LLMs to reason about hardware-oriented semantics, verify adherence to ISA specifications, and detect violations in execution behavior. This setting introduces unique challenges, such as handling long instruction sequences, limited contextual windows, and architecture-specific constraints, which existing LLM-based methods have not adequately addressed.

For instance, we used Llama-3.1-8B to generate test oracles for a given RISC-V instruction file. As shown in Fig. 2(a), when the instruction file is short (under 20 lines), Llama-3.1-8B successfully generated accurate test oracles. However, as shown in Fig. 2(b), when faced with instruction files containing more complex instruction logic and additional contextual information (more than 100 lines), Llama-3.1-8B struggles to fully comprehend the entire file and produces incorrect or confusing responses. Therefore, accurately tracking state changes and control flow when processing lengthy instruction files remains a challenge for LLMs when simulating instruction execution. They may lose the context of register

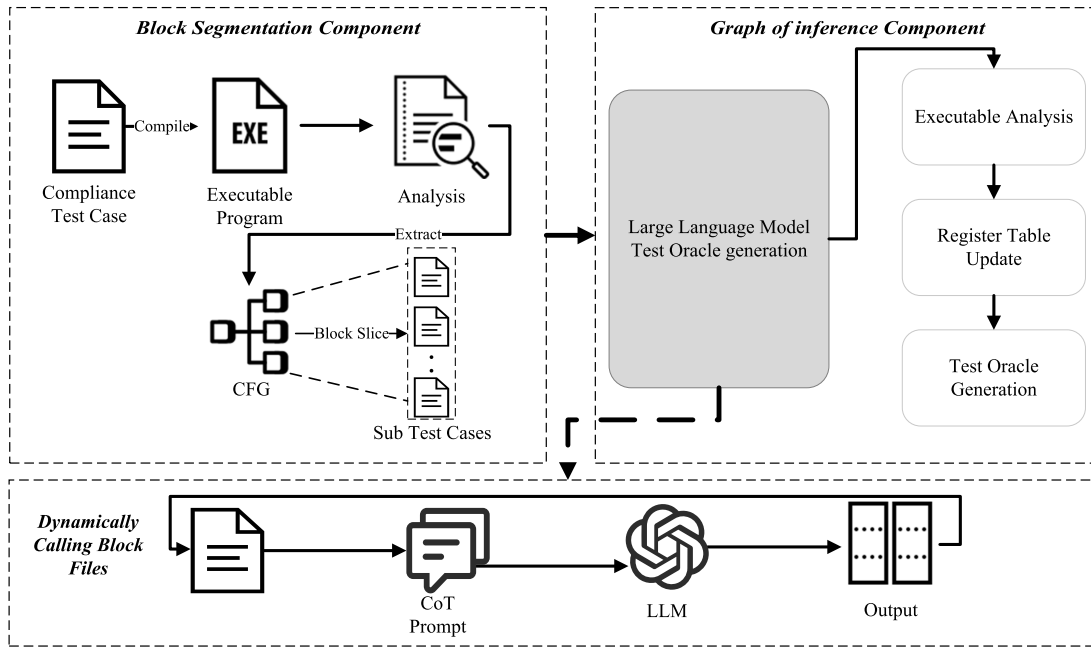


Fig. 3. Overview of TRAGIC.

values or fail to maintain consistency in memory operations across extended instruction sequences.

This observation motivates us to develop a hierarchical approach that decomposes complex instruction sequences into manageable subtasks at the basic block level, enabling LLMs to process each block independently while maintaining global consistency through structured state propagation. By hierarchically organizing the analysis, we can leverage the LLM’s reasoning capabilities within its effective context window and generate an accurate test oracle.

III. FRAMEWORK

In this section, we first present an overview of the TRAGIC framework and then explain its two major components: the BSC and GIC.

A. Overview

An overview of the TRAGIC framework is illustrated in Fig. 3. It takes a compliance test program as input and yields its test oracle with the help of LLMs as output. To facilitate accurate oracle generation, the framework follows a divide-and-conquer approach. BSC is responsible for the “divide” stage, while GIC handles the “conquer” stage. Recognizing that the complex control flows of test programs can pose significant challenges for LLMs, TRAGIC analyzes the control flow graph (CFG) of the test program and breaks it down into basic blocks. These basic blocks represent the nodes in the CFG and consist of straight-line sequences of instructions. Each basic block is further enhanced with the necessary context to function as a standalone RISC-V program. This way, it simplifies the processing units for the LLMs into shorter and more manageable subprograms. When navigating the simulation, LLMs will begin with the *entry* block. They are provided with clear instructions for analyzing and updating

the relevant program states of interest and determining the next block to which they should jump. To ensure better compliance with the instructions, CoT prompting methods are utilized. We present the detailed algorithm in Algorithm 1, with lines 4–19 for the BSC module and lines 20–32 for the GIC module. The quality of the test oracles generated by TRAGIC can be validated by comparing them with those generated with the help of Spike, not only in compliance testing, but also in other testing campaigns where Spike plays an indispensable role. Next, we elaborate on the designs for the two modules of TRAGIC.

B. Block Segmentation Component

BSC comprises two substeps: CFG analysis and Basic block segmentation.

1) *CFG Analysis*: To accurately model program execution behavior, BSC extracts and analyzes the CFG structure, which naturally reflects the execution order of instructions. We employ Angr to extract the initial CFG from RISC-V test cases, chosen for its rich functionality and seamless integration with our static analysis pipeline. Angr is a Python-based binary analysis framework that supports symbolic execution, static analysis, and dynamic analysis for automated vulnerability discovery and reverse engineering. However, the raw CFG requires additional processing to establish explicit mappings between basic blocks and instruction addresses. We therefore utilize objdump for disassembly, creating precise structural correspondences between the identified basic blocks and their ranges.

The extracted CFG can be formally represented as follows:

$$G = (V, E) \quad (1)$$

where V denotes the set of basic blocks and E represents directed edges indicating control flow transitions. Each basic

Algorithm 1 Workflow of TRAGIC

Input: RISC-V test case $\mathcal{T}$, Large Language Model $\mathcal{M}$
Output: Test oracle $\mathcal{O} = \{\sigma_1, \sigma_2, \dots, \sigma_n\}$

// Main Execution
1 $(G, \mathcal{V}, R) \leftarrow \text{BSC}(\mathcal{T})$ // Segment into basic blocks
2 $\mathcal{O} \leftarrow \text{GIC}(G, \mathcal{V}, R, \mathcal{M})$ // Generate oracle via inference
3 **return** $\mathcal{O}$

// Basic-block Segmentation Component
4 **Function** $\text{BSC}(\mathcal{T})$:
5 $G = (V, E) \leftarrow \emptyset$ // Initialize control flow graph
6 $\mathcal{B} \leftarrow \emptyset$ // Initialize block set
7 $R \leftarrow \mathbf{0}_{|V| \times |V|}$ // Initialize reference matrix
// Parse test case and identify leaders
8 $\text{leaders} \leftarrow \text{FindLeaders}(\mathcal{T})$
9 **foreach** $\ell \in \text{leaders}$ **do**
10 $v \leftarrow \text{CreateVertex}(\ell)$
// Extract instructions for this block
11 $b_v \leftarrow \text{ExtractBlock}(\mathcal{T}, \ell)$
12 $\mathcal{V} \leftarrow \mathcal{V} \cup \{b_v\}$
// Identify control flow edges
13 $\text{targets} \leftarrow \text{AnalyzeTargets}(b_v)$
14 **foreach** $t \in \text{targets}$ **do**
15 $E \leftarrow E \cup \{(v, t)\}$
16 $R[v][t] \leftarrow 1$
17 **end**
18 **end**
19 **return** $(G, \mathcal{V}, R)$

// Graph of Inference Component
20 **Function** $\text{GIC}(P = \{b_1, b_2, \dots, b_n\}, R, \mathcal{M})$:
21 $b^* \leftarrow b_0$
22 $\sigma \leftarrow \text{InitializeRegisters}()$
23 $\rho \leftarrow \langle b_0 \rangle$
24 **while** $\exists b \in P : R(b^*, b) = 1$ **do**
25 $\text{prompt} \leftarrow \text{BuildCoTPrompt}(b^*, \sigma)$
26 $\text{response} \leftarrow \mathcal{M}.\text{query}(\text{prompt})$
27 $(\sigma, \text{PC}, b') \leftarrow \text{ParseResponse}(\text{response})$
28 $\rho \leftarrow \rho \cup \langle b' \rangle$
29 $b^* \leftarrow b'$
30 **end**
31 $\mathcal{O} \leftarrow (\sigma, \text{PC}_{\text{new}}, \rho)$
32 **return** $\mathcal{O}$

block $b_i \in V$ is characterized by the following equation:

$$b_i = (\alpha_i, \beta_i, n_i) \quad (2)$$

with α_i as the entry address, β_i as the ending address, and n_i as the instruction count within the range $[\alpha_i, \beta_i]$. The edges are classified as follows:

$$E = E_s \cup E_c \quad (3)$$

where E_s represents sequential edges connecting consecutive blocks and E_c represents conditional edges for branch-dependent transitions.

To capture execution semantics and reference for oracle generation, BSC augments each basic block with semantic information

$$b_i^{\text{sem}} = (I_i, \omega_i, D_i) \quad (4)$$

where I_i represents the ordered instruction, ω_i denotes the terminating instruction determining exit behavior, and D_i captures register dependencies within the block. During execution,

only a subset of conditional edges will be activated

$$E_{\text{active}} \subseteq E_c \quad (5)$$

based on branching instructions (e.g., beq, bne, jal). Through this structured extraction and semantic analysis, we obtain the CFG of the RISC-V test case with full execution dependencies.

2) *Basic Block Segmentation*: After obtaining the CFG with full execution dependencies, TRAGIC performs block segmentation according to basic blocks b_i . This process transforms each basic block $b_i \in V$ into an independent instruction file, while preserving both computational logic and control flow integrity.

Specifically, TRAGIC maps the original RISC-V test case program P to a set of segmented files through

$$b_i \in P \rightarrow f_1, f_2, \dots, f_n \quad (6)$$

where $n = |V|$ is the number of basic blocks, and each file f_i corresponds to a basic block b_i . To maintain semantic completeness and interblock references, each segmented file f_i is structured as follows:

$$f_i = S_g \cup S_e \cup L_i \cup I_i \cup J_i \quad (7)$$

where S_g represents global symbol declarations ensuring external visibility of block b_i ; S_e consists of external symbol declarations for referencing other blocks; L_i marks the block's entry point with address α_i as a unique label; I_i preserves the ordered instruction sequence from b_i ; and J_i contains control transfer instructions based on the terminating instruction ω_i .

The interblock reference relationships are determined by the control flow edges extracted earlier. We define the reference matrix as follows:

$$R(b_i, b_j) = \begin{cases} 1, & \text{if } (b_i, b_j) \in E_s \cup E_c \\ 0, & \text{otherwise} \end{cases} \quad (8)$$

where E_s represents sequential edges (fall-through paths) and E_c denotes conditional edges (explicit branches). When $R(b_i, b_j) = 1$, the external symbol set S_e of block b_i must declare the label L_j of block b_j to preserve the linkage relationship.

For example, consider the RISC-V program shown in the left part of Fig. 4. The corresponding CFG G for this program consists of three basic blocks: b_{start} , b_{loop} , and b_{end} . The control flow transitions between these blocks are captured by the edge set

$$E = (b_{\text{start}}, b_{\text{loop}}), (b_{\text{start}}, b_{\text{end}}), (b_{\text{loop}}, b_{\text{loop}}), (b_{\text{loop}}, b_{\text{end}}) \quad (9)$$

where each edge represents a possible execution path from one block to another. Based on this edge set, the reference relationships between basic blocks can be represented as follows:

$$R = \begin{pmatrix} 0 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{pmatrix} \quad (10)$$

where rows and columns correspond to b_{start} , b_{loop} , and b_{end} , respectively, and each entry $R_{ij} = 1$ indicates that

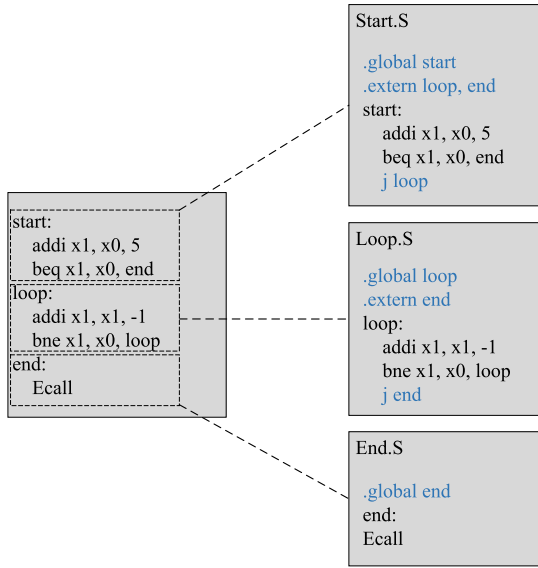


Fig. 4. Example of block segmentation of RISC-V test case.

block b_i contains a reference to block b_j , either explicitly through jump or branch instructions or implicitly through sequential execution. According to the reference matrix R , this RISC-V test case is decomposed into three modular files, each corresponding to one of the basic blocks, as shown in the right part of Fig. 4. Each file preserves the original instructions while incorporating necessary symbol declarations in the header to maintain interblock references. Through basic block segmentation, a RISC-V test case is transformed into multiple interreferenced files while preserving its original semantics. This decomposition explicitly restructures a complex, context-dependent task into smaller, modular subtasks, thereby enabling more structured and manageable analysis for subsequent processing stages.

C. Graph of Inference Component

The GIC systematically processes the segmented basic blocks from BSC through chain-of-thought (CoT) enhanced reasoning, enabling accurate oracle generation for RISC-V compliance testing. By leveraging explicit step-by-step reasoning, GIC transforms the complex task of full-program analysis into manageable block-level inference while maintaining global execution semantics.

Given the segmented test case $P = b_1, b_2, \dots, b_n$ and the reference matrix R from BSC, GIC initiates execution from the entry block b_0 and maintains a key register state table σ that tracks register values essential for program execution. For each block b_i , GIC constructs an explicit CoT prompt as shown in Fig. 5. The CoT reasoning strategy decomposes block-level analysis into four explicit steps. The process begins by parsing instructions in I_i to identify their operations and operands, establishing the block's computational structure. During sequential execution simulation, each instruction is processed in order while explicitly tracking register modifications to incrementally update the state σ_i . The LLM then analyzes the terminating instruction ω_i to determine branching conditions and target addresses, identifying whether control

You are analyzing a RISC-V instruction file.
Let's think through this step by step.

Input Information

- Current Block: <block_name>
- Instructions in this block:
- Current Register States:
- Previous Result: <previous_output>

Step 1: Instruction Parsing

- List each instruction with its operation type

Step 2: Sequential Execution Simulation

- Execute each instruction
- New values: [Calculate the new values]
- State Update: [Updated register state]

Step 3: Control Flow Analysis

- Branch condition: [Condition]
- Judgement Target Block

Step 4: Final State Determination

- Final register values:
- Final PC value
- Next block to execute

Final Output

- modified_registers: [register: value]
- final_pc: "0x..."
- next_block: [block name]

Fig. 5. CoT prompt for RISC-V instruction files.

follows a sequential edge in E_s or a conditional edge in E_c . Finally, the process consolidates all register modifications into the final state σ_i and computes the program counter value PC_i , yielding the complete state transformation for the block.

Through the output of reasoning step by step, GIC dynamically selects the next block b_{next} , utilizing the reference matrix R to validate the transition feasibility, where $R(b_i, b_{next}) = 1$. This iterative process continues, constructing an execution path

$$\rho = b_0 \xrightarrow{\sigma_0} b_1 \xrightarrow{\sigma_1} \dots \xrightarrow{\sigma_{k-1}} b_k \quad (11)$$

until reaching a terminal block b_k where $\forall j : R(b_k, b_j) = 0$. The execution path ρ not only captures the control flow trajectory but also maintains the register state evolution throughout the program execution.

Upon completing the iterative inference, GIC aggregates the final outputs to construct the comprehensive test oracle $\mathcal{O}$

$$\mathcal{O} = (\sigma_k, PC_k, \rho) \quad (12)$$

where $\mathcal{R}_k$ contains the terminal values of critical registers, PC_k represents the final program counter serving as a secondary validation mechanism, and ρ provides the complete execution trace. This multidimensional oracle captures both the computational results and the control flow behavior, addressing the complex requirements of RISC-V compliance testing. Furthermore, when the LLM encounters ambiguous instructions or complex branching logic, the explicit CoT reasoning trace enables targeted calibration by identifying specific reasoning steps that require refinement, thereby improving the robustness and reliability of oracle generation while minimizing computational overhead through focused interventions.

TABLE I
PERFORMANCE OF TRAGIC ON RISC-V OFFICIAL TEST SUITE BY INSTRUCTION EXTENSIONS

Extension	Description	Test Cases	Success (w/o GoI)	Success (with GoI)	Δ Success
I	Base integer instruction set	51	20	51	$\uparrow$ 31
M	Integer multiplication and division	13	2	13	$\uparrow$ 11
A	Atomic instructions	61	20	61	$\uparrow$ 41
F	Single-precision floating-point	39	11	39	$\uparrow$ 28
D	Double-precision floating-point	59	10	59	$\uparrow$ 49
C	Compressed instructions	48	21	48	$\uparrow$ 27
Other	Miscellaneous or mixed instructions	86	34	86	$\uparrow$ 52
Total		357	118	357	$\uparrow$ 239

IV. EXPERIMENTAL

In this section, four experiments are conducted to evaluate the effectiveness of TRAGIC. Specifically, our evaluation aims at answering the following research questions (RQs).

RQ1: Can TRAGIC generate a precise test oracle for RISC-V compliance testing?

RQ2: What is the performance of TRAGIC in Bug Detection?

RQ3: How well does TRAGIC generate test oracles compared to other methods?

RQ4: Can Graph of Inference improve the accuracy of the test oracle?

In RQ1, we primarily investigated whether TRAGIC can generate a precise test oracle of RISC-V compliance testing test cases. Through the RQ2, by integrating with the most advanced RISC-V instruction Fuzzer, we evaluate the performance of TRAGIC in bug detection. The RQ3 demonstrates TRAGIC's ability in generating RISC-V test oracles by comparing with other methods. And RQ4 focuses on the effect of the graph of inference in TRAGIC, we have performed an ablation study to illustrate this.

A. Studied Dataset

We conduct our experiment on the collection of handwritten RISC compliance test cases developed by the RISC-V Foundation [13]. This dataset contains a series of precompiled binary test programs in ELF format. These test programs are compiled with the RISC-V version of GCC. The test programs cover various instructions and implementations of the RISC-V ISA, like C-extensions, A-extensions, M-extensions, and F-extensions. The dataset includes both the test programs and their corresponding expected outputs. It is widely used by researchers and developers in the RISC-V community for testing and validation purposes. We downloaded this dataset and chose rev64i_m as our test cases. Because RV64 is the most widely used version of RISC-V, it is also the most commonly used version in the industry. Furthermore, we removed some unusable test cases, such as privileged mode test cases and temporarily unrated test cases, because they rely more on platform-specific page tables, which static analysis in software cannot obtain this information about specific hardware. After selection, there are a total of 357 test cases in the dataset, including C-extensions, A-extensions, M-extensions, I-extensions, and F-extensions.

B. Environment and Implementation

Our experiments were conducted on a server with a Ubuntu 20.04 64-bit system, Intel Core i9 CPU at 2.10 GHz, 120 GB of memory, and an NVIDIA A6000 GPU using CUDA 12.0. For LLM, we chose GPT-3.5-turbo [22] as our baseline LLM. For another open-source LLM like Llama-3.1-8B [23], CodeLlama-7B-hf [24], and CodeBERT-base [25], we do not choose to use them for they are not fine-tuned for RISC-V instructions. So that they cannot fully understand RISC-V instructions, especially the compliance testing cases, include more specific RISC-V instructions. We used the official API to access GPT-3.5-turbo and generate a test oracle.

C. RQ1: Can TRAGIC Generate Precise Test Oracle

Approach: We conduct an experiment in the official test dataset of the RISC-V Foundation. We used GPT-3.5-turbo [22] as our base LLM and ran all RISC-V files in the dataset to generate test oracles for each test case. Correspondingly, for all RISC-V files in the dataset, the official sources provided signature files that recorded the execution details and expected outputs. We then compared the analysis of these signature files with the test oracles we generated. If there was a match, it indicated that TRAGIC successfully generated the test oracle for the file. If not, it signified a failure in generating the appropriate test oracle.

Furthermore, to verify the effectiveness of our method, we reproduced all the errors officially recognized and scheduled for correction by Spike since September 2023, as recorded in the GitHub submissions. These errors totaled four. We opted not to include longer standing errors addressed by Spike, as they had been resolved and offered no referential value. Using our method, we generated test oracles for these four typical errors and compared them with the official expected outputs.

Results: The test oracle generation results for the official RISC-V test cases are presented in Table I. The results indicate that TRAGIC successfully generated test oracles for all 357 test cases in the dataset, achieving a 100% success rate. This demonstrates the effectiveness of our approach in generating accurate test oracles for RISC-V compliance testing. TRAGIC performed well across various instruction extensions, including I-Extension, M-Extension, A-Extension, F-Extension, D-Extension, C-Extension, and other miscellaneous instructions.

TABLE II
TEST ORACLE CORRECTNESS ACROSS DIFFERENT MODELS

Error Description	TRAGIC	GPT-4o	GPT-3.5-turbo
Inconsistent Instruction Decoding	FALSE	FALSE	FALSE
Floating-Point Load error	TRUE	FALSE	FALSE
Floating-Point Underflow	TRUE	FALSE	FALSE
Load Access Faults in Halfword	FALSE	FALSE	FALSE

We also applied our method to four real Spike error cases, with the results presented in Table II. TRAGIC successfully generated the correct test oracle for two of the errors. Specifically, in the two floating-point error cases where the test oracles were successfully generated, the LLM demonstrated strong understanding and reasoning capabilities. This success can be attributed to TRAGIC's block reasoning strategy, which effectively reduced the context length for these cases. Moreover, while the representation and operational rules of floating-point numbers are complex, they are relatively traceable. The IEEE 754 standard provides clear definitions for the format, rounding methods, and exception conditions of floating-point numbers. This well-defined structure is explicitly reflected in the code, making it easier for the model to capture and reason about the operations. Consequently, even an untuned LLM can accurately interpret and handle floating-point operations.

However, the LLM failed to generate correct test oracles for the other two error cases. These cases are influenced by specific user modes, where pure static code and general LLMs struggle to accurately simulate memory change and instruction operations under such specialized conditions.

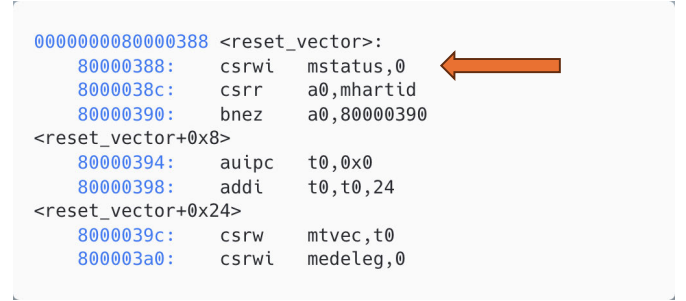
Conclusion: The experiments demonstrate the effectiveness of TRAGIC in generating test oracles for RISC-V test cases. In most test cases, LLM demonstrates its code understanding and inference capabilities.

D. RQ2: What Is the Performance of TRAGIC in Bug Detection?

Approach: We conducted an experiment to evaluate the performance of TRAGIC in bug detection by combining two advanced RISC-V instruction fuzzers: DifuzzRTL and Cascade. We reproduce these two fuzzer tools with the source code provided by their works and use their default configurations. Specifically, we used the test cases generated by these fuzzers as input for TRAGIC. These test cases were then simulated within real-world RISC-V RTL designs, namely Rocket-Chip and Boom, which are among the most widely used RISC-V processor designs in real-world environments. The test oracles generated by TRAGIC were compared with the expected outputs from the RTL designs. Any mismatches would indicate the potential presence of a bug in the RISC-V processor designs. If a mismatch was detected, we proceeded to reproduce the test case to further verify whether it indeed represents a genuine bug. We conducted this experiment in one week by continuously running the test cases generated by the fuzzers and the test oracles generated by TRAGIC. In this process, about 10 000 test cases were generated by the fuzzers, and we used TRAGIC to generate test oracles for all of them.

TABLE III
BUG DETECTION RESULTS OF TRAGIC

Bug Description	Bug Location	Bug Status
CSRWI configuration instruction error	Boom	Unknown
EBREAK timeout instruction error	Rocket-chip	UnKnown
Discrepancy in Mode CSR	Boom	Known
Invalid rm field warning	Boom	Known
fsqrt calculation exception	Boom	Known
fddiv division by zero exception	Boom	Known

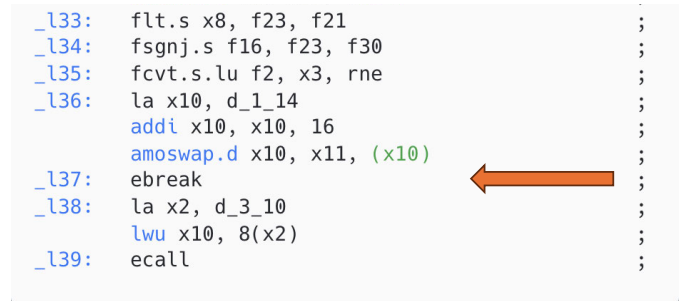


```

0000000080000388 <reset_vector>:
80000388:  csrwi  mstatus, 0
8000038c:  csrr   a0, mhartid
80000390:  bnez   a0, 800000390
<reset_vector+0x8>
80000394:  auipc  t0, 0x0
80000398:  addi   t0, t0, 24
<reset_vector+0x24>
8000039c:  csrwi  mtvec, t0
800003a0:  csrwi  medeleg, 0

```

Fig. 6. Case study 1 in boom processor about CSRWI.



```

_l33:  flt.s  x8, f23, f21
_l34:  fsgnj.s f16, f23, f30
_l35:  fcvt.s.l f2, x3, rne
_l36:  la     x10, d_1_14
      addi x10, x10, 16
      amoswap.d x10, x11, (x10)
_l37:  ebreak
_l38:  la     x2, d_3_10
      lwu  x10, 8(x2)
_l39:  ecall

```

Fig. 7. Case study 2 in boom processor about ebreak.

Results: The results of the experiment are shown in Table III. TRAGIC successfully identified six bugs in two widely used RISC-V processor designs. Two of these bugs were previously unknown, while the other four had been known and reported by previous researchers.

Case Study 1: As shown in Fig. 6, the test case is a CSRWI instruction that writes to the CSR register. In the Boom core, `csrwi mstatus, 0` is an instruction used to clear the `mstatus` register, which is a core control register in the RISC-V system. It involves critical system states such as interrupt enable and privilege level in machine mode. The clearing operation of this register is usually used to initialize or reset the state. However, the execution of this instruction in the Boom simulation environment did not produce the expected results, resulting in simulation errors.

Case Study 2: As shown in Fig. 7, in the Boom core, when a virtual address error (such as a page fault) occurred after executing the `ebreak` instruction, the system failed to write the faulting address to the specified register. While the exception triggered by the `ebreak` instruction itself does not directly involve a memory access error, it is crucial for the virtual address causing the exception to be stored. This information is essential for the exception handler to determine how to

handle the page fault or memory access error. Spike, however, is unable to detect these errors, as it times out while attempting to process them.

Conclusion: The experimental results demonstrate that TRAGIC is capable of effectively detecting bugs in RISC-V processor designs. By leveraging the test oracles generated by TRAGIC, we were able to identify both known and unknown bugs, showcasing the potential of our approach in enhancing the reliability and correctness of RISC-V processors.

E. RQ3: How Well Does TRAGIC Generate Test Oracles Compared to Other Method

Approach: Due to the lack of sufficient high-quality compliance test cases, we did not select TOGLL as our baseline method, as TOGLL relies on a fine-tuned LLM. Furthermore, concepts such as test signatures, commonly used in unit tests, do not exist in the context of an ISA. Consequently, it is not feasible to fully reproduce TOGLL for the RISC-V test oracle generation task. Building on this foundation, we chose to compare TRAGIC with the traditional few-shot learning method in the official Compliance testing dataset. Specifically, we employed the few-shot approach to fine-tune the LLM for generating test oracles. Meanwhile, we also compared TRAGIC with zero-shot learning, which is a common method for generating test oracles. In the process of few-shot fine-tuning, we were constrained by the context length limit of the LLM. Therefore, we selected three representative examples from C-Extensions, providing detailed descriptions of their functions along with their corresponding final test oracles as input to the LLM. By applying prompt engineering, we guided the LLM to output test oracles for new test cases based on these examples.

Results: As shown in Table IV, compared to zero-shot learning, few-shot learning cannot improve the performance of LLM. In most cases, few-shot learning has the same success rate as zero-shot learning. This is because the few-shot learning method is not able to provide sufficient context or examples for the LLM to effectively learn from, leading to similar performance as zero-shot learning. Only in a few test cases of C-Extension, the few-shot learning method can slightly improve the performance of LLM. This is because we selected examples from the C-Extension to perform few-shot learning for the LLM. But the success rate of few-shot learning is still below 50%.

Furthermore, the performance of few-shot learning and zero-shot learning is not as good as TRAGIC. For the majority of failed test cases in few-shot learning and zero-shot learning, their length exceeded the LLM's context limit, and segmented inputs resulted in the loss of contextual information. The Block segmentation of TRAGIC effectively mitigated this issue by breaking down the input into smaller, manageable segments. This approach allows the model to maintain a coherent understanding of the instruction flow and data dependencies, ultimately leading to more accurate test oracle generation.

Conclusion: Our comparative evaluation demonstrates that TRAGIC consistently outperforms both the few-shot and zero-shot methods. By slicing the instruction code into smaller code segments based on the CFG, TRAGIC addresses the issue

TABLE IV
SUCCESS RATE ACROSS EXTENSION CATEGORIES FOR FEW-SHOT AND ZERO-SHOT METHODS

Extension Category	Few-shot Success	Zero-shot Success
I-Extension	0.40	0.40
M-Extension	0.15	0.15
A-Extension	0.32	0.32
F-Extension	0.28	0.28
D-Extension	0.17	0.17
C-Extension	0.46	0.38
other	0.36	0.36

of context information loss caused by excessively long input lengths.

F. RQ4: Can Graph of Inference Improve the Accuracy of the Test Oracle

Approach: To prove the effect of GIC, we conducted an experiment to compare the accuracy of the test oracle generated by TRAGIC and TRAGIC without GIC. Specifically, we removed the GIC from the TRAGIC pipeline. We input the CFG generated by Angr and the RISC-V test case into LLM to generate the test oracle. Subsequently, we compared the accuracy of the test oracle generation between the LLM in the absence of GIC and the complete TRAGIC framework.

Results: Table I presents the performance of TRAGIC without the GIC. The results indicate a significant drop in accuracy, with only 118 successful test cases out of 357, resulting in a success rate of less than 40%. Particularly in facing long test cases in D-Extension and M-Extension, LLM can only generate 2 and 10 test oracles, respectively. This is primarily due to the LLM's limited context window, which restricts its ability to effectively process and reason about longer sequences of instructions. As the number of instructions increases, the model may struggle to maintain coherence and accuracy in its reasoning, leading to a higher failure rate.

Furthermore, the experimental results also indicate that merely feeding the CFG into the LLM does not significantly enhance its ability to understand instruction programs. After providing the CFG to the LLM, the accuracy of the generated test oracles shows no substantial improvement compared to methods like few-shot and zero-shot learning. Therefore, TRAGIC employs the GIC to better assist the LLM in comprehending the overall control flow of the instruction set program, thereby enabling the generation of accurate test oracles.

Conclusion: Experimental results underscore the necessity of our Graph of Inference approach in decomposing complex instruction behaviors and guiding the reasoning process.

V. THREATS TO VALIDITY

A. Internal Threats

The internal threats to the TRAGIC method primarily stem from correctness of block segmentation. The correctness of the block segmentation is crucial for the LLM to accurately understand the instruction program. If the block segmentation is not correctly generated, it may lead to incorrect reasoning and ultimately affect the accuracy of the test oracle generation. To mitigate this risk, we employed a combination of static

analysis and dynamic execution to ensure that the block segmentation is generated correctly. We also conducted extensive testing and validation to verify the correctness of the block segmentation.

B. External Threats

The external threats to the TRAGIC method primarily arise from the selection of the test case suite. We utilized the manually written test case suite provided by the official RISC-V community, which aims to cover most of the basic usage scenarios of the RISC-V instruction set. However, due to the complexity and diversity of the RISC-V instruction set, there might still exist some edge cases or extreme scenarios that are not fully covered by this test suite. To mitigate this potential coverage insufficiency issue, we adopted a complementary strategy. We specifically focused on test cases that can cause the Spike simulator to produce errors or exhibit abnormal behavior. By validating the effectiveness of the TRAGIC method on these challenging test cases, we can enhance its coverage capability for complex situations and boundary conditions.

Furthermore, we acknowledge that the block segmentation phase can become increasingly expensive as instruction files grow larger. This is particularly evident due to the performance limitations of Angr, which is implemented in Python and not optimized for high-throughput static analysis. However, it is important to note that our use case focuses on compliance-oriented test inputs, which are typically concise and structurally simple. Under these constraints, the runtime overhead introduced by Angr remains within an acceptable range. To further improve the scalability, we also plan to explore alternative static analysis frameworks such as Ghidra, Dyninst, and LLVM IR-based tools. These solutions offer the potential for faster control-flow extraction while preserving the necessary precision for downstream segmentation.

VI. RELATED WORK

A. Test Oracle Generation

Automatically generating effective test oracles has consistently been a challenging task. To address the test oracle problem, several approaches have commonly been used, such as implicit oracles, which have relied on runtime-enforced checks to detect errors, like null pointer dereference exceptions; differential oracles, which have compared the outputs of two programs or versions [26]; and metamorphic oracles, which have validated known relationships between input mutations and the corresponding output changes [27], [28].

To fully realize their potential, test generation techniques have needed to move beyond implicit, differential, and metamorphic oracles by associating test inputs with precise, input-specific oracle assertions that accurately reflect the intended program behavior. Researchers have investigated using pattern-matching methods to create test oracles derived from code comments and textual documentation [29], [30], [31], [32]. These methods could have inferred assertion oracles, which have compared actual program outputs to expected results, as well as exception oracles that have captured the

expected exceptional behaviors of the program under test. More recently, natural language processing (NLP) techniques have been employed to generate test oracles, utilizing a transformer-based model that has learned patterns from the methods under test and developer-authored test cases [9], [11]. Dinella et al. [11] introduced TOGA, a Transformer-based test oracle generator, which analyzes the context of test cases to generate the test oracle of Java unit tests. In a study using the Defects4J [33], TOGA identified 57 bugs, of which 30 were unique and undetected by any competing methods. Furthermore, Hossain et al. [12] proposed TOGLL, a method for generating Java unit test oracles by leveraging a fine-tuned LLM combined with a specified context approach. Experimental results demonstrate that TOGLL outperforms TOGA in identifying real bugs from the Defects4J dataset. However, since TOGLL has relied on recurring Java code patterns in the training data to infer and generate expected test assertions, it could not have been directly applied to test oracle generation for assembly instructions. Consequently, there has remained a significant need for a sophisticated and mature methodology for generating test oracles of RISC-V compliance testing.

B. Large Language Model

LLMs has been predominantly developed using transformer architectures [34], [35], [36], [37]. These models have marked significant advancements in NLP by providing models with extraordinary language comprehension capabilities and the ability to generate contextually relevant and semantically coherent text. LLMs are typically pretrained on extensive textual data from diverse sources such as books, articles, websites, and even source code. Recent studies [38], [39] have underscored the capabilities of LLMs in code recognition, understanding, and generation. For example, Chen et al. [40] have developed ChatTester, a novel approach that leverages the CoT reasoning capabilities of LLMs for unit test generation. Plein et al. [41] have introduced a new method based on bug reports to generate test cases using LLM. Leinonen et al. [39] have explored the potential of LLMs in generating code explanations. All these methods have shown LLM's potential to transform various aspects of software development and maintenance.

Furthermore, Ding et al. [20] proposed TRACED, a method for fine-tuning code language models to enhance code understanding and execution path analysis through pretraining tasks. TRACED addresses the limitations of traditional code language models, such as CodeX and UnixCoder, which struggle to accurately predict code execution paths. By constructing diverse pretraining tasks, TRACED achieves significant improvements in code execution path prediction, program state prediction, and program coverage prediction. However, TRACED is primarily designed for the C programming language, and there remains a substantial difference between procedural programming languages and ISAs. More importantly, the lack of sufficient pretraining data for RISC-V compliance test oracle generation poses a major challenge. In addition to TRACED, Pei et al. [42], [43] have proposed a series of Transformer-based models for learning and understanding binary program execution. Similarly, in the context

of program understanding, Souza and Pradel [44] introduced LExecutor, a tool designed to predict missing values during program execution. These studies highlight the remarkable potential of LLMs in code understanding and learning tasks. Nevertheless, the unique characteristics of RISC-V instructions necessitate further exploration of feasible methods to enable LLMs to effectively learn and understand RISC-V-specific features and semantics.

VII. CONCLUSION AND FUTURE WORK

In this article, we propose TRAGIC, an LLM-based method for generating test oracles in RISC-V compliance testing. TRAGIC demonstrates its capability to generate precise and comprehensive test oracles for RISC-V test cases. By combining the BSC and the GIC, TRAGIC effectively utilizes the LLM to produce accurate and detailed test oracles. The effectiveness of TRAGIC has been validated through experiments using the manually written test case suite provided by the official RISC-V community. Furthermore, TRAGIC has been successfully applied to detect bugs in real-world RISC-V processor designs, showcasing its practical applicability and effectiveness in enhancing the reliability of RISC-V compliance testing. Nevertheless, we also recognize that completely eliminating blind spots in test coverage is an ongoing challenge. Despite the effectiveness of TRAGIC in generating accurate test oracles for the existing test suite, there may still be corner cases or rare scenarios that are not fully covered. To address this issue, future work can explore the integration of automated test generation techniques, such as fuzz testing and mutation testing, to supplement manually written test cases. By leveraging these techniques, we can generate a more diverse and comprehensive set of test cases that exercise different parts of the RISC-V instruction set and processor implementations. Furthermore, new ISA extensions may also have potential impacts on TRAGIC. Therefore, through automated test case generation techniques, we can also explore generating sufficient test cases for fine-tuning the base LLM to mitigate the impact of new ISA extensions.

REFERENCES

- [1] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanovic, "The RISC-V instruction set manual, volume I: User-level ISA, version 2.0," Dept. EECS, Univ. California, Berkeley, CA, USA, Tech. Rep. UCB/EECS-2014-54, 2014, p. 4.
- [2] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanovic, "The risc-v instruction set manual, volume I: Base user-level ISA," Dept. EECS, UC Berkeley, Berkeley, CA, USA, Tech. Rep. UCB/EECS-2011-62, 2011.
- [3] V. Herdt, D. Große, and R. Drechsler, "Towards specification and testing of RISC-V ISA compliance?" in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2020, pp. 995–998.
- [4] (2024). *Forvis_RISCV-ISA-Spec*. [Online]. Available: https://github.com/rsnikhil/Forvis_RISCV-ISA-Spec
- [5] RISC-V Software Source. (2025). *Spike: A RISC-V ISA Simulator*. Accessed: Jun. 26, 2025. [Online]. Available: <https://github.com/riscv-software-src/riscv-isa-sim>
- [6] V. Herdt, D. Große, H. M. Le, and R. Drechsler, "Verifying instruction set simulators using coverage-guided fuzzing," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2019, pp. 360–365.
- [7] F. Solt, K. Ceasay-Seitz, and K. Razavi, "Cascade: CPU fuzzing via intricate program generation," in *Proc. 33rd USENIX Secur. Symp. (USENIX Secur.)*, Jul. 2024, pp. 5341–5358.
- [8] J. Hur, S. Song, D. Kwon, E. Baek, J. Kim, and B. Lee, "DifuzzRTL: Differential fuzz testing to find CPU bugs," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2021, pp. 1286–1303.
- [9] M. Tufano, D. Drain, A. Svyatkovskiy, and N. Sundaresan, "Generating accurate assert statements for unit test cases using pretrained transformers," in *Proc. IEEE/ACM Int. Conf. Autom. Softw. Test (AST)*, May 2022, pp. 54–64.
- [10] C. Watson, M. Tufano, K. Moran, G. Bavota, and D. Poshyvanyk, "On learning meaningful assert statements for unit test cases," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. (ICSE)*, Oct. 2020, pp. 1398–1409.
- [11] E. Dinella, G. Ryan, T. Mytkowicz, and S. K. Lahiri, "TOGA: A neural method for test Oracle generation," in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng. (ICSE)*, May 2022, pp. 2130–2141.
- [12] S. B. Hossain and M. Dwyer, "TOGLL: Correct and strong test Oracle generation with LLMs," 2024, *arXiv:2405.03786*.
- [13] (2024). *RISCV-Tests*. [Online]. Available: <https://github.com/riscv/riscv-tests>
- [14] G. J. Myers, T. Badgett, and T. M. Thomas, *The Art of Software Testing*. Chichester, U.K.: Wiley, 2004.
- [15] S. B. Hossain, M. B. Dwyer, S. Elbaum, and A. Nguyen-Tuong, "Measuring and mitigating gaps in structural testing," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, May 2023, pp. 1712–1723.
- [16] G. Jahangirova, D. Clark, M. Harman, and P. Tonella, "Test Oracle assessment and improvement," in *Proc. 25th Int. Symp. Softw. Test. Anal.*, Jul. 2016, pp. 247–258.
- [17] M. Staats, M. W. Whalen, and M. P. E. Heimdahl, "Programs, tests, and Oracles: The foundations of testing revisited," in *Proc. 33rd Int. Conf. Softw. Eng. (ICSE)*, May 2011, pp. 391–400.
- [18] M. Whalen, G. Gay, D. You, M. P. E. Heimdahl, and M. Staats, "Observable modified condition/decision coverage," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, May 2013, pp. 102–111.
- [19] Y. Zhang and A. Mesbah, "Assertions are strongly correlated with test suite effectiveness," in *Proc. 10th Joint Meeting Found. Softw. Eng.*, Aug. 2015, pp. 214–224.
- [20] Y. Ding, B. Steenhoek, K. Pei, G. Kaiser, W. Le, and B. Ray, "TRACED: Execution-aware pre-training for source code," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Feb. 2024, pp. 1–12.
- [21] J. Lee et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, Jul. 2022, pp. 24824–24837.
- [22] (2024). *GPT-3.5-Turbo*. [Online]. Available: <https://platform.openai.com/docs/models/gpt-3.5-turbo>
- [23] (2024). *Llama-3.1-8B*. [Online]. Available: <https://huggingface.co/meta-llama/Llama-3.1-8B>
- [24] (2024). *Codellama-7B-HF*. [Online]. Available: <https://huggingface.co/codellama/Codellama-7b-hf>
- [25] (2024). *Codebert-Base*. [Online]. Available: <https://huggingface.co/microsoft/codebert-base>
- [26] W. M. McKeeman, "Differential testing for software," *Digital Tech. J.*, vol. 10, no. 1, pp. 100–107, 1998.
- [27] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Trans. Softw. Eng.*, vol. 42, no. 9, pp. 805–824, Sep. 2016.
- [28] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," *ACM SIGPLAN Notices*, vol. 49, no. 6, pp. 216–226, Jun. 2014.
- [29] A. Blasi et al., "Translating code comments to procedure specifications," in *Proc. 27th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2018, pp. 242–253.
- [30] A. Goffi, A. Gorla, M. D. Ernst, and M. Pezzè, "Automatic generation of Oracles for exceptional behaviors," in *Proc. 25th Int. Symp. Softw. Test. Anal.*, Jul. 2016, pp. 213–224.
- [31] R. Pandita, X. Xiao, H. Zhong, T. Xie, S. Oney, and A. Paradkar, "Inferring method specifications from natural language API descriptions," in *Proc. 34th Int. Conf. Softw. Eng. (ICSE)*, Jun. 2012, pp. 815–825.
- [32] S. B. Hossain, A. Filieri, M. B. Dwyer, S. Elbaum, and W. Visser, "Neural-based test Oracle generation: A large-scale evaluation and lessons learned," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2023, pp. 120–132.
- [33] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A database of existing faults to enable controlled testing studies for Java programs," in *Proc. Int. Symp. Softw. Test. Anal.*, Jul. 2014, pp. 437–440.
- [34] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *Google*, vol. 21, no. 140, pp. 1–67, 2022.

- [35] H. Touvron et al., “LLaMA: Open and efficient foundation language models,” 2023, *arXiv:2302.13971*.
- [36] A. Zeng et al., “GLM-130B: An open bilingual pre-trained model,” 2022, *arXiv:2210.02414*.
- [37] A. Zeng et al., “ChatGLM: A family of large language models from GLM-130B to GLM-4 all tools,” 2024, *arXiv:2406.12793*.
- [38] J. White, S. Hays, Q. Fu, J. Spencer-Smith, and D. C. Schmidt, “ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design,” in *Generative AI for Effective Software Development*. Cham, Switzerland: Springer, 2024, pp. 71–108.
- [39] J. Leinonen et al., “Comparing code explanations created by students and large language models,” in *Proc. Conf. Innov. Technol. Comput. Sci. Educ. V. I*, Jun. 2023, pp. 124–130.
- [40] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, “ChatUniTest: A framework for LLM-based test generation,” 2023, *arXiv:2305.04764*.
- [41] L. Plein, W. C. Ouedraogo, J. Klein, and T. F. Bissyandé, “Automatic generation of test cases based on bug reports: A feasibility study with large language models,” in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng., Companions*, Apr. 2024, pp. 360–361.
- [42] K. Pei et al., “StateFormer: Fine-grained type recovery from binaries using generative state modeling,” in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Aug. 2021, pp. 690–702.
- [43] K. Pei et al., “NeuDep: Neural binary memory dependence analysis,” in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2022, pp. 747–759.
- [44] B. Souza and M. Pradel, “LExecutor: Learning-guided execution,” in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2023, pp. 1522–1534.



Zhihao Xu received the M.S. degree in computer science and technology from Dalian Maritime University, Dalian, China, in June 2024. He is currently pursuing the joint Ph.D. degree with Southeast University, Nanjing, China, and Monash University, Melbourne, VIC, Australia, under the joint supervision of Prof. Bixin Li and Prof. Xiaoning Du.

His research interests include AI for software engineering (AI4SE), open-source software engineering, system software security, and intelligent software testing.



Bixin Li (Member, IEEE) received the bachelor's and master's degrees in mathematics from Anhui University, Hefei, China, in 1991 and 1994, respectively, and the Ph.D. degree in software engineering from Nanjing University, Nanjing, China, in 2001.

He is currently a Full Professor with the School of Computer Science and Engineering, Southeast University, Nanjing, where he is the Head of the Software Engineering Institute, and works with more than 50 young people on software architecture and blockchain security projects. He has authored or

co-authored more than 180 research articles and patented more than 80 inventions in China. His research interests include program analysis, software testing and verification, software safety, and security techniques.

Dr. Li is the Chair of the Technology Committee of Software Engineering Standards of Jiangsu Province, China.



Xiaoning Du (Member, IEEE) received the bachelor's degree from Fudan University, Shanghai, China, in 2014, and the Ph.D. degree from Nanyang Technological University, Singapore, in 2020.

She is a Senior Lecturer (equivalent to U.S. Associate Professor) at the Faculty of Information Technology, Monash University, Melbourne, VIC, Australia. Her research primarily focuses on the security and quality assurance of intelligent software systems, with a particular emphasis on intelligent software engineering tools. She has published more

than 40 articles in top-tier conferences and journals.

Dr. Du's work has received or been nominated for the ACM SIGSOFT Distinguished Paper Award multiple times. She also received the 2024 FIT Dean's Early Career Researcher of the Year Award and the 2024 Google Research Scholar Award.



Lulu Wang received the bachelor's degree in computer science and the Ph.D. degree in software engineering from Southeast University, Nanjing, China, in 2006 and 2012, respectively.

He is an Associate Professor at the School of Computer Science and Engineering, Southeast University. His research interests include path profiling, program analysis, and program slicing.



Li Liao received the Ph.D. degree from The Hong Kong Polytechnic University, Hong Kong, in 2009.

She is a Lecturer at the School of Computer Science and Engineering, Southeast University, Nanjing, China. Her research interests include trustworthy cloud service, software measurement, and software evolution.



Ying Zhou is a Lecturer at the School of Computer Science and Engineering, Southeast University, Nanjing, China. Her research interests include software modeling and verification, cyberphysical systems, and formal methods.